



# Contents

|          |                                                                |          |
|----------|----------------------------------------------------------------|----------|
| <b>1</b> | <b>Operator overloading</b>                                    | <b>3</b> |
| 1.1      | Section overview . . . . .                                     | 3        |
| 1.2      | What is operator overloading? . . . . .                        | 3        |
| 1.2.1    | Example . . . . .                                              | 3        |
| 1.2.2    | What operators can be overloaded? . . . . .                    | 4        |
| 1.2.3    | Some basic rules . . . . .                                     | 4        |
| 1.2.4    | Examples . . . . .                                             | 4        |
| 1.2.5    | The class we will be using through out this section . . . . .  | 5        |
| 1.3      | Overloading the assignment operator . . . . .                  | 6        |
| 1.3.1    | Overloading the copy assignment operator (deep copy) . . . . . | 6        |
| 1.3.2    | Example . . . . .                                              | 7        |
| 1.4      | Overloading the move assignment operator . . . . .             | 8        |
| 1.4.1    | Overloading the move assignment operator . . . . .             | 8        |
| 1.4.2    | Overloading the move assignment operator - steps . . . . .     | 8        |
| 1.4.3    | Example . . . . .                                              | 9        |

# Chapter 1

## Operator overloading

### 1.1 Section overview

- Operator overloading allows the user to use the operator with objects.
- Overloading operators as member functions.
- Overloading operators as global functions.
- Overloading stream insertion (<<) and extraction operators (>>).

### 1.2 What is operator overloading?

- Using traditional operators such as +, =, \*, etc. with user-defined types.
- Allows user defined types to behave similar to built-in-types.
- Can make code more readable and writeable.
- Not done automatically (except for the assignment operator). Operator overloading operations must be explicitly defined.

#### 1.2.1 Example

- Suppose we have a Number class that models any number, so we can do anything with numbers such as complex numbers, integers, decimals, etc.
- For this kind of thing we would need to make this class call functions or methods to multiply - for example - and this does not feel natural.

```
1 Number result = multiply(add(a, b), divide(c, d));  
2 Number result = (a.add(b)).multiply(c.divide(d));
```

- For C++ it does not make sense to do operations with two objects, thus you need to tell the C++ compiler what you mean when you are doing an operation with two objects, you can do this with methods - like exemplified above - or with overloading operators.
- Only overload operators when it makes sense, otherwise the code is not going to be readable and calling a function/method is better.
- Lets overload the +, \* operators.

```
1 Number result = (a + b) * (c /d);
```

- Overloading operators is just for easier programming, behind the scenes we are still calling functions and methods.

### 1.2.2 What operators can be overloaded?

- The majority of C++'s operators can be overloaded.
- The following operators cannot be overloaded:

|        |                           |
|--------|---------------------------|
| ::     | Scope resolution operator |
| ?:     | Conditional operator      |
| .*     | member operator           |
| .      | dot operator              |
| sizeof | sizeof operator           |

### 1.2.3 Some basic rules

- Precedence and associativity cannot be changed. You cannot make operators so that one operation takes place before another when the operators do not behave like that normally.
- 'arity' cannot be changed (i.e. can't make the division operator unary), this means that if the operator expected to do something with two arguments then when you overload it, it expects two arguments. Example:  $a + b$  expects argument a and argument b, you cannot overload it so that it takes in a third or takes in just one, it remains at 2.
- Can't overload operators for primitive type (`int`, `double`, etc).
- Can't create new operators.
- `[]`, `()`, `->`, and the assignment operator (`=`) must be declared as member methods.
- Other operators can be declared as member methods or global functions.

### 1.2.4 Examples

|                                                                    |                                                                                                       |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <pre>1 int a = b + c; 2 a &lt; b; 3 std::cout &lt;&lt; a;</pre>    | <pre>1 std::string s1 = s2 + s3; 2 s1 &lt; s2; 3 std::cout &lt;&lt; s1;</pre>                         |
| <pre>1 double a = b + c; 2 a &lt; b; 3 std::cout &lt;&lt; a;</pre> | <pre>1 std::string my_string = s2 +    ↪ s3; 2 s1 &lt; s2; 3 s1 == s2; 4 std::cout &lt;&lt; s1;</pre> |

### 1.2.5 The class we will be using through out this section

Class definition in Mystring.h:

```
1 #ifndef MYSTRING_H
2     #define MYSTRING_H
3
4 class Mystring {
5 private:
6     char *str;
7 public:
8     Mystring();
9     Mystring(const char *s);
10    Mystring(const Mystring &source);
11    ~Mystring();
12
13    void display() const;
14    int get_length() const;
15    const char *get_str() const;
16 };
17 #endif
```

Class implementation in Mystring.cpp:

```
1 // no args constructor.
2 Mystring::Mystring() : str(nullptr) {
3     // Make an empty string
4     str = new char[1];
5     *str = '\0';
6 }
7
8 // copy constructor.
9 Mystring::Mystring(const char *s) : str{nullptr} {
10     if (s == nullptr) {
11         str = new char[1];
12         *str = '\0';
13     }
14     else {
15         str = new char[strlen(s) + 1];
16         strcpy(str, s);
17     }
18 }
19
20 // destructor.
21 Mystring::~Mystring() {
22     delete[] str;
23 }
24
25 // print the string.
26 void Mystring::display() const {
27     std::cout << str << ": " << get_length() << std::endl;
28 }
29
30 // string length.
31 int Mystring::get_length() const { return strlen(str); }
```

```

32
33 // string getter.
34 const char *Mystring::get_str() const { return str; }

```

## 1.3 Overloading the assignment operator

- We will overload the assignment operator to mean deep copy from a string object into another string object.
- C++ provides a default assignment operator used for assigning one object to another.

```

1 Mystring s1 {"frank"};
2 Mystring s2 = s1; // NOT assignment it is the same as Mystring s2 {s1}; this is
  ↳ done my constructors to initialize.
3 s2 = s1; // assignment, once initialized the assignment can be called.

```

- Default is memberwise assignment (shallow copy):
  - If we have raw pointer data member we must use deep copy.

### 1.3.1 Overloading the copy assignment operator (deep copy)

- The assignment must be overloaded as a member function. So we provide a member prototype:

```
Type &Type::operator=(const Type &rhs);
```

- Replace “Type” with the name of the class in question.

```

1 Mystring &Mystring::operator=(const Mystring &rhs);
2 s2 = s1; // we write this.
3 s2.operator=(s1); // operator= method is called.

```

- Behind the scenes the method “.operator=” is called.

Overloading the copy assignment operator (deep copy) step by step.

```

1 Mystring &Mystring::operator=(const Mystring &rhs) {
2     if (this == &rhs) {
3         return *this;
4     }
5     delete[] str;
6     str = new char[std::strlen(rhs.str) + 1];
7     std::strcpy(str, rhs.str);
8     return *this;
9 }

```

- First, check for self assignment, if the right hand side (rhs) is exactly the same as the left hand side (lhs) then we terminate the function by returning `this` since we already made the copy. We check the address of both pointers, if they are the same, then we assigning an object to itself.

```

1 if (this == &rhs) { // p1 = p1;
2     return *this; // return current object.
3 }

```

- Deallocate storage for `this->str` since we are overwriting it. Since we need to deallocate this data because we will allocate the string somewhere else. This is to avoid memory leaks.

```
1 delete[] str;
```

- Allocate storage for the deep copy.

```
1 str = new char[std::strlen(rhs.str) + 1];
```

- Perform the copy. Use `strcpy()` function to copy the string one character at a time.

```
1 std::strcpy(str, rhs.str);
```

- Return the current by reference to allow chain assignment. Remember chain assignment is when we have several assignments in the same line (example: `s1 = s2 = s3 = s4;`)

```
1 return *this; // s1 = s2 = s3 = s4;
```

### 1.3.2 Example

Now lets add the overloaded assignment operator to the `Mystring` class. This is in `Mystring.h`.

```
1 #ifndef MYSTRING_H
2     #define MYSTRING_H
3     class Mystring {
4     private:
5         char *str;
6     public:
7         Mystring();
8         Mystring(const char *s);
9         Mystring(const Mystring &source);
10        ~Mystring();
11
12        // copy assignment (overloaded).
13        Mystring &operator=(const Mystring &rhs);
14
15        void display() const;
16        int get_length() const;
17        const char *get_str() const;
18    };
19 #endif
```

On `Mystring.cpp` I add this implementation.

```
1 Mystring &Mystring::operator=(const Mystring &rhs) {
2     std::cout << "Copy assignment" << std::endl;
3     if (this == &rhs) {
4         return *this;
5     }
6     delete[] this->str;
7     str = new char[std::strlen(rhs) + 1];
8     std::strcpy(this->str, rhs.str);
9     return *this;
10 }
```

On the main.cpp:

```
1 int main() {
2     Mystring a {"hello"};
3     Mystring b;
4     b = a;
5     b = "This is a test.";
6     return 0;
7 }
8 /* OUTPUT:
9 Copy assignment
10 Copy assignment
11 */
```

## 1.4 Overloading the move assignment operator

- Move assignment operator is also denoted with the '='.
- You can choose to overload the move assignment operator.
  - C++ will use the copy assignment operator if necessary.

```
1 Mystring s1;
2 s1 = Mystring {"Frank"}; // move assignment, this is an unnamed object that is
   ↳ constructed and has no name (this is an r value), by moving it to s1 it is
   ↳ granted a name.
```

- The assignment deep copy operator in the last section works with left value references, so values. The move assignment works with right value references. This means temporary unnamed objects that need assignment (move).
- You don't need to provide a move assignment operator, if you don't C++ will use the copy constructor by default. It is always recommended to provide a move assignment operator and a move constructor.
- The main difference between the move assignment operator and the copy assignment operator (seen in the last section) is that in one we are deep copying every character - in this case - of the string, while in this one we are just stealing the pointer from the right hand side and assigning it to the left.

### 1.4.1 Overloading the move assignment operator

- `Type &Type::operator=(Type &&rhs);`

```
1 // in the Mystring class.
2 Mystring &Mystring::operator=(Mystring &&rhs);
3 s1 = Mystring{"Joe"}; // move operator= called.
4 s1 = "Frank"; // move operator= called.
```

### 1.4.2 Overloading the move assignment operator - steps

```
1 Mystring &Mystring::operator=(Mystring &&rhs) {
2     if (this == &rhs) { // self assignment.
3         return *this;    // return current object.
4     }
5 }
```



```

5   delete[] str; // deallocate current storage.
6   str = rhs.str; // steal the pointer
7   rhs.str = nullptr; // null out the rhs object.
8   return *this; return *this;
9 }

```

- Check for self assignment:

```

1  if (this == &rhs) {
2      return *this;
3  }

```

- Deallocate storage for `this->str` since we are overwriting it and we don't want a memory leak.

```

1  delete[] str;

```

- Steal the pointer from the rhs object and assignment it to `this->str`:

```

1  str = rhs.str;

```

- Null out the rhs pointer:

```

1  rhs.str = nullptr;

```

- Return the current object by reference to allow chain assignment.

```

1  return *this;

```

### 1.4.3 Example

Comparing the move assignment operator and the copy assignment operator. On the `Mystring.h`:

```

1  #include <iostream>
2  #include <vector>
3  #include <string.h>
4
5  #ifndef MYSTRING_H
6      #define MYSTRING_H
7
8  class Mystring {
9  private:
10     char *str;
11 public:
12     Mystring(); // no args constructor.
13     Mystring(const char *s); // overloaded constructor.
14     Mystring(const Mystring &source); // copy constructor
15     ~Mystring(); // destructor
16     Mystring(Mystring &&source); // move constructor
17
18     Mystring &operator=(const Mystring &rhs); // copy assignment.
19     Mystring &operator=(Mystring &&rhs); // Move assignment.
20
21     void display() const;
22     int get_length() const;

```

```

23     const char *get_str() const;
24 };
25
26 // no args constructor.
27 Mystring::Mystring() : str(nullptr) {
28     // Make an empty string
29     str = new char[1];
30     *str = '\0';
31 }
32
33 // copy constructor.
34 Mystring::Mystring(const char *s) : str{nullptr} {
35     if (s == nullptr) {
36         str = new char[1];
37         *str = '\0';
38     }
39     else {
40         str = new char[strlen(s) + 1];
41         strcpy(str, s);
42     }
43 }
44
45 // destructor.
46 Mystring::~Mystring() {
47     delete[] str;
48 }
49
50 // move constructor
51 Mystring::Mystring(Mystring &&source) : str{source.str} {
52     source.str = nullptr;
53     std::cout << "Move constructor used" << std::endl;
54 }
55
56 // move assignment.
57 Mystring &Mystring::operator=(Mystring &&rhs) {
58     std::cout << "Using move assignment" << std::endl;
59     if (this == &rhs) {
60         return *this;
61     }
62     delete[] str;
63     str = rhs.str;
64     rhs.str = nullptr;
65     return *this;
66 }
67
68
69 // print the string.
70 void Mystring::display() const {
71     std::cout << str << ": " << get_length() << std::endl;
72 }
73
74 // string length.
75 int Mystring::get_length() const { return strlen(str); }
76

```

```

77 // string getter.
78 const char *Mystring::get_str() const { return str; }
79
80 // copy assignment operator overloading.
81 Mystring &Mystring::operator=(const Mystring &rhs) {
82     if (this == &rhs) {
83         return *this;
84     }
85     delete[] str;
86     str = new char[strlen(rhs.str) + 1];
87     strcpy(str, rhs.str);
88     return *this;
89 }
90 #endif

```

On the main.cpp:

```

1  #include <iostream>
2  #include <vector>
3  #include "Mystring.h"
4
5  using namespace std;
6
7  int main() {
8      Mystring a{"Hello"}; // overloaded constructor
9      a.display();
10     a = Mystring{"Hola"}; // overloaded constructor
11     a.display();
12     a = "Bonjour";
13     a.display();
14     return 0;
15 }
16
17 /* OUTPUT:
18 Hello: 5
19 Using move assignment
20 Hola: 4
21 Using move assignment
22 Bonjour: 7
23 */

```